

Characterizing Tensor-Parallel Crossover in vLLM Serving on Dual NVIDIA Tesla T4 GPUs

Mohammad Waqas

Abstract

Adding a second GPU to a language-model server divides model work and memory but also introduces synchronization, collective communication, scheduling, and process overhead. We characterize the conditions under which two-way tensor parallelism (TP2) improves upstream vLLM serving on a pinned, PHB-connected pair of NVIDIA Tesla T4 GPUs in Kaggle. The principal experiment compares TP1 and TP2 across four serving-ready model families, three exact-token workload shapes, six concurrency levels, and five matched logical repetitions per model–workload combination. Of 60 terminal logical shards and 720 planned fresh-server cells, 55 shards yielded canonical performance outcomes and five terminated at a frozen resource boundary. The first sustained favorable output-throughput point under the predeclared paired-repetition 95% confidence-interval rule varied by model and workload: prefill-heavy work crossed at concurrency 4 for Llama, Phi, and Ministral; their balanced and short workloads crossed later. Qwen crossed at concurrency 64 for balanced work, showed no sustained short-workload crossover, and resource-gated TP2 prefill-heavy work at concurrency 64. An independent two-rank NCCL microbenchmark characterizes communication on the same topology but does not isolate its causal contribution to serving. The kaggle-vllm toolkit delivers and records a checksum-pinned upstream runtime; it does not replace the inference engine. A separate ALLaM-7B Arabic/English case study illustrates descriptive high-concurrency throughput and context-capacity differences while retaining its weaker within-server uncertainty boundary. These findings describe the tested runtime, workload, and resource envelope rather than a universal TP2 speedup.

1 Introduction

Tensor parallelism is an attractive response to a small per-device memory budget. It can partition weights and intermediate work so a model or context

that cannot execute on one device may execute on two. Yet a second GPU is not a free throughput multiplier. A request can incur additional collective communication, launch coordination, worker scheduling, and distributed-runtime work at each generation step. The benefit depends on how much computation is exposed, how many requests can be batched, the model’s architecture, and the memory envelope. An operational TP2 run and a performance win are distinct observations. Prior serving work establishes batching, KV-cache management and distributed execution, but it does not determine the model–workload–concurrency TP2 crossover for this particular constrained dual-T4 PHB configuration.

This paper asks: **under which model, token-shape, and concurrency conditions does TP2 overcome its overhead on a PHB-connected dual-Tesla-T4 upstream-vLLM stack?** The platform is useful as a constrained systems case. Each T4 has limited memory for contemporary instruction models, and the observed GPU-to-GPU topology is PHB rather than NVLink. A single-GPU configuration can be viable, faster at low load, or unable to fit a particular context. Those outcomes must be represented separately. A zero-throughput value for a blocked configuration would be a scientific error because no valid performance observation exists.

The work follows a staged evidence sequence. M1 established operational TP and measured low-load OPT-125M behavior. M2 explored concurrency with Qwen2.5-3B in one serving matrix. M3 measured two-rank NCCL all-reduce independently. M4 is the principal repeated, multi-model serving experiment with compatibility gating, exact-token workloads, and a frozen resource guard. A later PR #29 ALLaM-7B study is a supplementary bilingual serving case with its own protocol and evidential limits. The M2 Qwen threshold is a useful motivating observation; it is not the final Qwen estimate because the accepted M4 repetitions have higher authority.

Our contributions are empirical and reproducibility oriented. First, we document a lightweight method for staging an immutable upstream vLLM CUDA wheel on a validated Kaggle dual-T4 host while preserving the host’s PyTorch/CUDA installation. Second, we provide a controlled TP1/TP2 comparison across four principal model families, three exact-token workload shapes, six concurrency points, and matched logical repetitions. Third, we show that the sustained favorable output-throughput point varies jointly with model and workload, and that a resource-gated condition must be reported as capacity evidence. Fourth, we measure two-rank communication as an independent system characteristic without treating it as proof of the serving mechanism. Fifth, a bounded ALLaM case separates bilingual execution, descriptive online performance, and context capacity.

The result is a characterization of one pinned upstream-vLLM stack. It does not introduce an inference engine, PagedAttention, tensor parallelism, NCCL, or a checkpoint format. It does not evaluate answer quality, other GPU interconnects, production traffic, or the performance of arbitrary vLLM versions. The paper retains failures and provenance recoveries alongside favorable cells because these change the interpretation of the experiment.

2 Background and related work

2.1 Autoregressive serving and outcome dimensions

An autoregressive server first processes the prompt and then generates output tokens through repeated decode steps. Prefill and decode stress the system differently: prompt processing exposes a larger arithmetic workload, whereas successive decode steps may have smaller per-step work and frequent synchronization. Concurrent requests permit batching but also raise scheduling pressure, memory demand, and client-observed latency. A throughput point therefore depends on both a request’s token shape and the offered concurrency. The client records output tokens per wall-clock second and request throughput separately. Time to first token (TTFT), time per output token (TPOT), inter-token latency (ITL), and end-to-end (E2E) latency represent different stages and user experiences. Lower latency at one metric does not imply a higher output-token rate, and higher capacity does not imply faster service. We distinguish **compatibility** (the pinned stack initializes and executes), **capacity** (the requested configuration fits the frozen resource envelope), **performance** (rate and latency conditional on execution), and **quality** (the usefulness or correctness of generated answers). This study measures the first three, not model quality; Arabic and English text generation in the ALLaM case establishes execution, not linguistic competence.

Orca introduced iteration-level scheduling and selective batching for generative serving [13]. vLLM developed block-managed KV-cache allocation and PagedAttention to reduce cache waste and support efficient serving [5]. These systems explain why scheduler and memory behavior matter in addition to raw model compute. This study uses upstream vLLM rather than replacing those components. The original vLLM evaluation used its own hardware and workloads; we make no direct performance comparison to those reported results. Sarathi studies chunked prefill and decode-maximal batching [2], while DistServe studies separating prefill and decode placement under latency objectives [14]. Those papers alter scheduling or placement; here the scheduler and engine remain upstream vLLM, and the variable of interest

is TP degree within one frozen client, token shape and PHB-connected T4 platform.

2.2 Tensor parallelism and communication

Tensor/model parallelism partitions selected layer computations over ranks and requires data exchange between them. Megatron-LM is a primary reference for practical intra-layer model parallelism [12]. NCCL defines the two-rank all-reduce operation used in our independent communication measurement: each rank receives the reduction of rank inputs [9]. The cost of the actual serving graph need not equal a microbenchmark’s all-reduce cost. It also includes model-specific graph structure, kernel choices, batch state, synchronization, worker orchestration, and client behavior. We therefore use M3 to bound discussion of communication, not to assign a causal fraction of each serving latency.

2.3 Platform and measurement principles

The historical environment comprises two Tesla T4 SM75 devices connected through an observed PHB path, Python 3.12.13, PyTorch 2.10.0+cu128, CUDA toolkit 12.8.93, NCCL 2.27.5, and historical NVIDIA driver 580.159.04. The driver-reported CUDA maximum and the CUDA runtime/toolkit are different fields. Table 1 gives the retained machine identity. A recovered October collector independently validates post-study runtime reconstruction on driver 580.178.04, without revising historical M1–M4 hardware fields.

MLPerf Inference motivates explicit scenarios, metrics, and reproducible configuration descriptions, although this work is not an MLPerf submission [11]. ACM artifact review principles motivate traceable sources, executable checks, and clear separation between artifact availability and independently validated results [1]. We claim neither an ACM badge nor external independent replication. The precise client and frozen prompts remain part of the measured system, and the study’s comparisons are internal to that system.

3 kaggle-vllm system and reproducibility design

The kaggle-vllm v0.2.0 package is a compatibility and runtime-delivery toolkit around upstream vLLM. Its profile records a native wheel filename, immutable Hugging Face binary-repository revision, and expected SHA256 digest. The observed upstream source lineage is vLLM commit `a26e8dc7ff2111a005144d775ecf9cebf56c45b2`; the native distribution identifies as `0.18.2.dev`

Table 1: Historical benchmark hardware and pinned runtime (M1–M4; October validation is separate).

Component	Observed value
GPU	2 x Tesla T4, SM75, 15360 MiB each
Topology	PHB
Driver	580.159.04
Python	3.12.13
PyTorch	2.10.0+cu128
CUDA toolkit	12.8.93
Torch CUDA	12.8
NCCL	2.27.5
Native vLLM distribution	0.18.2.dev0+ga26e8dc7f.d20260822.cu128

0+ga26e8dc7f.d20260822.cu128. The pinned wheel digest is 5a9bd710b8a19fdd23abb3442baad892da977466f996334decd533a225f5fd0c, and the binary repository revision is f6b4f10de54924ed6fe9e28cceab84eca7276ab6. These are identity and provenance fields, not a claim that kaggle-vllm authored upstream vLLM algorithms.

The bootstrap’s staging design uses `pip -target -no-deps` for the native wheel and a separate dependency overlay. It verifies the wheel checksum before staging, checks a validated profile against host properties, and avoids resolving or replacing the host’s PyTorch/CUDA stack. The stage and overlay paths are explicit, and strict doctor/bootstrap checks can fail closed when the host does not match the profile. The paper’s validation is based on retained manifests and source, not on a new GPU execution performed for this writing task. The package is neither an official Kaggle product nor an official vLLM distribution. Its system contribution here is repeatable runtime delivery and evidence provenance.

Research evidence is retained at several levels: raw request and resource ledgers, notebook and source identities, runtime manifests, SHA256SUMS files, review records, accepted reconciliation, deterministic analysis JSON, generated tables and figures, and manuscript prose. The hierarchy matters. A text summary may be stale after a later accepted shard. In this project, accepted machine-readable M4 analysis and terminal state outrank the existing manuscript, while the exploratory ALLaM P3 notebook never supersedes the final P4 machine table. The copied canonical tables and figures in this paper project are byte-identical to their sources; their paths and hashes are recorded in provenance CSVs. No model weight, native wheel, or Qwen sharded-state archive is redistributed here.

One historical Qwen TP2 sharded-state archive was verified as a separate persistence/reload capability. Its size is 4,773,220,584 bytes and its sidecar SHA256 is 12dcb264cb74e6fa2947b5f1fbebfa14562afa2292387f49e447b3290bc0b83b; the archive hash check passed during this audit. M4 did not benchmark that archive: its cells loaded regular pinned model checkpoints. The sharded-state format belongs to upstream vLLM, and this earlier artifact is neither an M4 input nor an algorithm contribution.

Frozen notebook / M4 shard identity → kaggle-vllm 0.2.0 profile,
checksum and guards → upstream vLLM TP1 or TP2 on the T4 pair
→ checksummed logs, request metrics and telemetry.

Figure 1: Experimental and evidence architecture. Derived from the canonical SVG; the source diagram is preserved byte-identically in figures/canonical.

4 Research questions

RQ1: At which tested concurrency point, if any, does TP2 first achieve a sustained favorable output-throughput result for each model and exact-token workload? **RQ2:** How do TTFT, TPOT, ITL, E2E latency, and request throughput behave relative to output throughput? **RQ3:** How do prefill-heavy, balanced, and short/decode-heavy shapes change the observed regimes? **RQ4:** Which pinned models pass the SM75 compatibility gate, and which cells are constrained by the frozen resource envelope? **RQ5:** What can the independent NCCL/PHB measurement tell us about communication, and what causal questions remain unresolved? **RQ6:** Does a separate ALLaM-7B Arabic/English serving case illustrate the distinction between low-load behavior, higher-concurrency throughput, and context capacity without being treated as another M4 repetition?

5 Experimental methodology

5.1 Historical hardware, runtime, and model population

The benchmark used the retained dual-T4 Kaggle environment described in Table 1. Both GPUs are SM75 devices; the topology record shows a PHB path, not an NVLink token. Five checkpoint families entered M4 compatibility gating. Four passed TP1 and TP2 serving readiness: Qwen/Qwen2.5-3B-Instruct, meta-llama/Llama-3.2-3B-Instruct, microsoft/Phi-4-mini-instruct,

and mistralai/Minstral-3-3B-Instruct-2512-BF16. Their short immutable revisions are in Table 2, with complete repository IDs and 40-character revisions in Appendix A and the canonical model metadata CSV. The five-model population and its source identities were recorded in `research/model_matrix.json` on 2026-09-07 and bound by `research/M4_SOURCE_FREEZE.json` with status `PREPARED_FOR_KAGGLE_NO_GPU_RESULTS`. Compatibility gating preceded the serving matrix: four checkpoints became eligible for principal rates, while Gemma 3 4B remained a recorded negative gate rather than being replaced after seeing TP results. The retained planning file documents pinned checkpoint identity, architecture, precision and resource estimates, but does not state a broader model-sampling rationale. Gemma 3 4B failed the frozen FP16/SM75 dtype-intersection gate before serving readiness. This negative result applies to the tested stack and dtype; it does not say Gemma is generally incapable of T4 inference, and no Gemma 4 checkpoint substitutes for it.

5.2 Exact-token workloads, cells, and metrics

The principal shapes are short (128 input, 64 output tokens), balanced (512 input, 256 output tokens), and prefill-heavy (2048 input, 128 output tokens). Their prompt construction and manifest checks enforce actual token counts rather than relying on intended text length. Prefix caching was disabled for M4. Each eligible model-workload combination was tested at offered concurrency 1, 4, 8, 16, 32, and 64 under TP1 and TP2. Five logical repetitions were frozen. The design yields $4 \text{ models} \times 3 \text{ shapes} \times 6 \text{ concurrency points} \times 2 \text{ TP degrees} \times 5 \text{ repetitions} = 720$ planned fresh-server cells in 60 model-workload-repetition shards. A fresh-server cell means a newly initialized vLLM server process for one TP/concurrency condition, as required by `research/KAGGLE_EXECUTION_CHECKLIST.md` and the execution plan. It does not imply a new physical Kaggle allocation. M3 instead uses fresh-process communication repetitions; ALLaM P4 waves share one server lifetime. The resulting request observations are nested inside M4 cells; they are not independent experimental replicates.

Output throughput is the completed output-token rate measured by the retained client. Request throughput is successful requests per second. TTFT, TPOT, ITL, and E2E latency come from the repository’s client/analysis definitions; they are client-specific rather than direct GPU kernel times. Resource ledgers sample per-GPU memory and utilization and record request failures. Sampled maxima are not guaranteed unsampled physical peaks. Comparisons remain within each model and exact workload; there is no across-

model quality or architecture-efficiency ranking because the checkpoints, tokenizers, and tasks differ.

5.3 Experimental unit and inference rule

The inferential unit for M4 is a matched logical repetition. At a given model, workload, and concurrency, TP1 and TP2 repetition means form paired differences. The canonical analyzer’s favorable cell rule requires the paired-repetition mean-delta 95% confidence interval to exclude zero in the favorable direction. The final sustained crossover is the first favorable point for which all higher frozen concurrency points remain favorable. Thus a small positive mean ratio, such as one without a favorable paired CI, is not a sustained crossover. Five repetitions, not hundreds of requests, determine the inferential count. The paper uses the project’s exact retained analysis and does not substitute a new stochastic CI procedure.

The cell outcome also includes compatibility and resource states. A frozen 14.5 GiB = 14,848 MiB = 15,569,256,448 byte ceiling applies independently to each physical GPU in M4. The execution guide and checklist document the exact binary-unit conversion and independent device checks; they do not give a further empirical or safety-margin derivation for choosing 14.5 GiB. We therefore report it as a predeclared protocol threshold rather than infer a post-hoc rationale. When a cell crosses that guard, the corresponding performance measure is unavailable. It is neither throughput zero nor necessarily a CUDA out-of-memory exception. No post-hoc relaxation or sixth repetition is introduced to manufacture a favorable point. The terminal reconciliation has 55 canonical performance shards, five reviewed resource-gated shards, zero unresolved planned shards, and no remaining normal continuation.

5.4 Chronology, provenance, and limits of control

M1, M2, and M3 used distinct protocols and establish different propositions. M1 validates operational tensor-parallel execution and low-load overhead. M2 is a single Qwen serving matrix, useful for locating a potential crossover but inadequate as the final repeated estimate. M3 is an independent communication microbenchmark. M4 applies the frozen model/workload/concurrency matrix and matched-repetition rule. Review and reconciliation files record known source issues, including r03 Ministral notebook provenance recovery and a V19 pre-bootstrap Llama integrity abort before a corrected continuation. These records are retained rather than silently treated as ordinary successful runs. The optional M5 independent client/simulator cross-check

was not run; the resulting conclusions are specific to this measurement client and platform.

5.5 Acceptance, aggregation and claim traceability

The accepted result is not simply the newest file by timestamp. The M4 execution reconciliation identifies each logical shard’s terminal state, including preservation, resource-gate review and any remaining work. The assembled principal raw JSON links accepted shards to the deterministic analysis JSON; the latter contains full cell-level distributions, paired differences and crossover classifications. The generated CSVs flatten these fields for reading and plotting. A claim about a model/workload crossover is checked against the accepted analysis and summary, not inferred from a plotted line or copied from manuscript prose. The source hierarchy and SHA256 manifests also prevent a later notebook narrative from silently replacing a preserved cell. This paper project’s claim ledger records the exact field or row used for every numerical headline and for capacity/compatibility statuses.

The analysis does not reclassify failed or guarded executions as low performance. The compatibility gate answers whether the pinned checkpoint/runtime combination reaches serving. The resource gate answers whether a requested cell fits the frozen allocation ceiling. Only accepted performance cells enter TP1/TP2 rate and latency comparisons. This sequencing matters because performance is conditional on both compatibility and capacity: a blank speed cell after a gate is not evidence against TP2’s computational efficiency. The manuscript preserves the terminology `RESOURCE_GATED` from the accepted evidence and reserves “OOM” for a recorded CUDA out-of-memory condition. Similarly, success of an individual smoke prompt does not certify readiness for every concurrency point.

The historical runtime record and October post-study validation are kept in separate strata. The recovered collector, evidence ZIP, exact PR #29 merge tarball and standalone runtime JSON were hash-verified after an earlier unsuccessful Downloads search. The October runtime confirms post-study reconstruction under driver 580.178.04, but a later driver or allocation is not a retroactive benchmark condition. The recovered outer ZIP is not byte-identical to the notebook-reported original; its internally checksummed payloads verify. The separate PR #29 P2 runtime is not an October collector substitute. Every new file made for this paper is under the sibling project, and the original data are never repaired in place.

Table 2: Principal M4 models and revision prefixes; full immutable revisions appear in Appendix A.

Model	Hugging Face ID	Revision prefix
Qwen2.5-3B	Qwen/Qwen2.5-3B-Instruct	aa8e725379
Phi-4-mini	microsoft/Phi-4-mini-instruct	cfbefacb99
Ministral-3-3B	mistralai/Ministral-3-3B-Instruct-2512-BF16	b6d637bef2
Llama-3.2-3B	meta-llama/Llama-3.2-3B-Instruct	0cb88a4f76

Table 3: Frozen exact-token workload shapes; each uses concurrency 1, 4, 8, 16, 32, 64.

Workload	Input tokens	Output tokens	Logical repetitions
short	128	64	5
balanced	512	256	5
prefill_heavy	2048	128	5

6 M1: operational TP and low-load overhead

M1’s role is operational and diagnostic. A successful TP2 run establishes that the pinned runtime can initialize distributed workers on the two physical T4 devices and produce output. It does not show that two GPUs improve serving. The OPT-125M comparison makes this distinction visible under low load. In graph mode, the mean output throughput was 1921.42 tokens/s for TP1 and 1408.55 for TP2 over five trials per configuration. Under eager execution, the corresponding means were 312.19 and 172.33 tokens/s. Table 4 retains the sample standard deviations and full precision in the source CSV. Both modes favor TP1 in this small-model diagnostic. The substantial graph/eager difference also shows why execution mode is an experimental variable, not an interchangeable implementation detail.

The OPT-125M evidence has a narrower population and workload than M4. Its small compute demand can make overhead conspicuous, but one cannot transfer its rate ratio directly to 3B-class models. The proper inference is that TP2 functionality and TP2 benefit must be tested separately. That observation motivates the later concurrency matrix. M1’s Qwen batching checks likewise belong to operational validation rather than a final estimate of a sustained serving crossover. No claim is made about model answer quality from generated smoke text.

M1 also helps define the artifact standard used later. The retained directory contains the individual JSON trials, logs, topology record, environment and provenance records, a summary, and a SHA256 manifest. During this

audit the manifest verified against its files. The experiment is thus inspectable beyond a narrative claim, but the five trials and one small model are still insufficient to characterize all serving regimes. M1 is reported before M2–M4 because it records the initial tension between compatibility and speed that the main study was designed to resolve.

Table 4: M1 OPT-125M low-load output throughput; values are trial means and sample standard deviations.

Mode	TP	Mean output tok/s	SD	Trials
graph	1	1921.42	24.45	5
graph	2	1408.55	40.40	5
eager	1	312.19	6.58	5
eager	2	172.33	1.58	5

7 M2: initial Qwen concurrency crossover

M2 changed the question from a low-load diagnostic to a single Qwen2.5-3B-Instruct serving matrix. Offered concurrency ranged over 1, 4, 8, 16, 32, and 64 for TP1 and TP2. The machine CSV records output throughput, request success, sampled memory and selected latency percentiles for each point. All rows in the retained single matrix report zero failed requests and no observed CUDA OOM. The output-throughput curve first favors TP2 at C=16: TP1 measured 138.75 and TP2 174.27 output tokens/s. At C=32 the rates were 158.65 and 267.79, and at C=64 they were 177.82 and 311.94 tokens/s. Table 5 displays all six matched concurrency points; the associated canonical figure makes the initial crossover visually clear.

The M2 observation was exploratory. It is one matrix, not five independent logical repetitions, and its prompt shape, execution chronology, and conditions are not the full M4 multi-model design. It motivates the hypothesis that enough concurrent work can amortize TP2 overhead. It does not establish C=16 as a universal Qwen threshold or even as the final repeated Qwen balanced threshold. In M4, the accepted Qwen balanced summary first satisfies the sustained paired-repetition 95% CI rule at C=64. The apparent conflict disappears when the two source authorities and criteria are kept explicit: M2 reports a single-run mean advantage; M4 reports an inferentially supported sustained point under its frozen matrix. We retain both observations and do not silently edit either measurement.

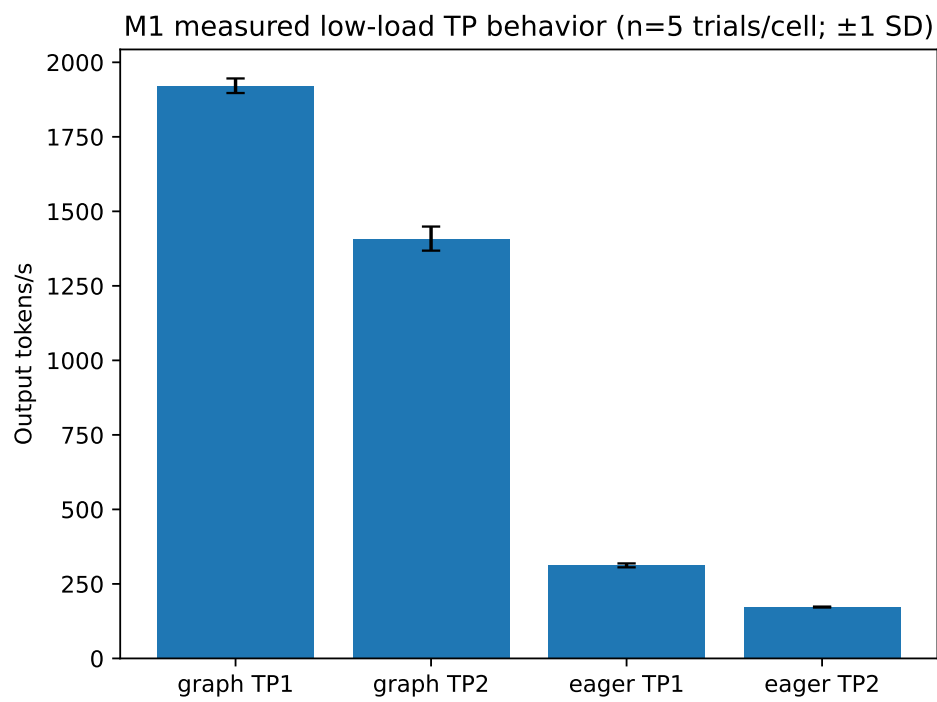


Figure 2: M1 low-load OPT-125M comparison; copied canonical figure.

The M2 latency columns further caution against treating a throughput curve as the complete serving result. TP2 can reduce TTFT at one point yet increase TPOT at another, and a high-concurrency output-token rate says little about isolated request latency. These client metrics are workload-specific; they are not direct kernel timings. The M2 result therefore justifies the broad metric panel in M4, including TTFT, TPOT, ITL, E2E latency, request throughput, failure outcomes, and sampled resource use.

Table 5: Exploratory single-matrix M2 Qwen output throughput; M4 governs final crossover inference.

Concurrency	TP1 tok/s	TP2 tok/s
1	26.29	14.95
4	76.87	55.21
8	108.53	99.80
16	138.75	174.27
32	158.65	267.79
64	177.82	311.94

8 M3: measured two-rank NCCL/PHB communication

M3 independently measured raw two-rank NCCL all-reduce on the same class of PHB-connected T4 environment. Ten payload sizes were tested, with five fresh-process repetitions and 100 timed collectives per payload and repetition. The resulting 5,000 critical-path observations use the maximum rank time for a collective, not the faster rank alone. The raw ledger, environment, fit JSON, residuals and review records are retained; their SHA256 manifest verified during this audit. The fit is a description of the tested communication configuration, not a model of the entire vLLM serving step.

The accepted linear form is $\mathbf{T_us(S)} = \mathbf{112.135079\ microseconds} + \mathbf{S / 4.063916\ GB/s}$, where S is the measured effective payload quantity in the analyzer’s convention. The fitted intercept’s 95% interval is 93.111210–131.158948 microseconds, and the effective beta interval is 4.051054–4.076860 GB/s. The canonical fit reports $R^2 = 0.999984836$ and $RMSE = 20.136460$

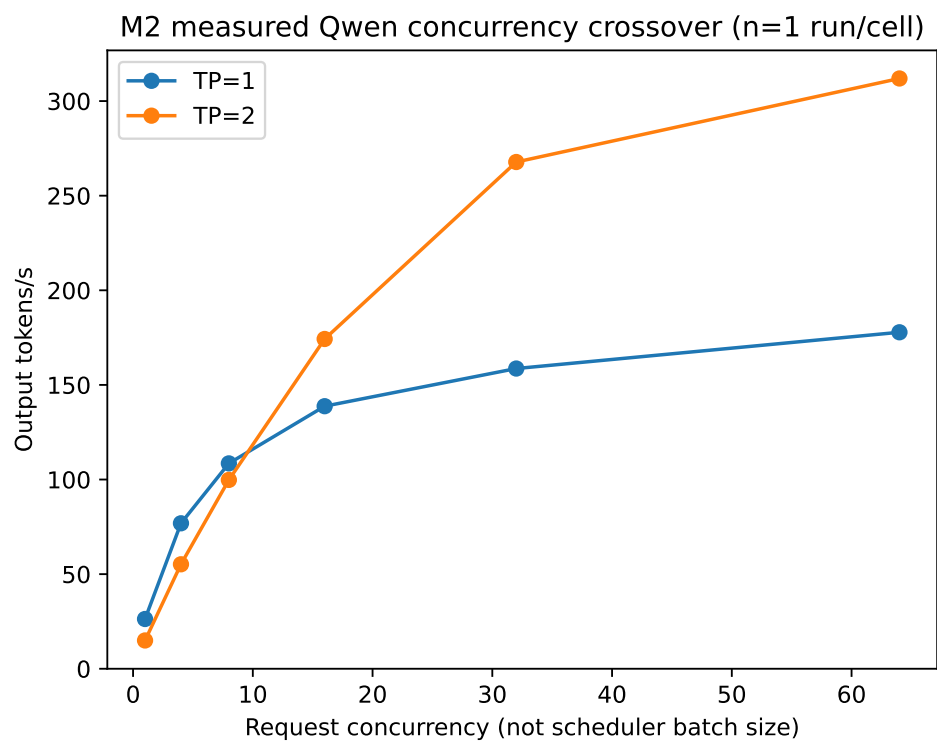


Figure 3: M2 exploratory Qwen single-matrix crossover; copied canonical figure.

microseconds. At a 64 MiB payload the observed effective payload bandwidth is 4.030741 GB/s. The ten payload summaries and residual figure allow readers to inspect how a high aggregate R^2 can coexist with nontrivial small-message variation. Table 6 records the primary fit fields without replacing the project’s statistical method.

The fitted intercept is **not** classical transport alpha, universal PCIe/PHB latency, or a per-layer serving synchronization charge. It includes the measured collective path and the specifics of process, GPU, NCCL version, payload convention, and timing method. The effective beta is similarly configuration-specific. M3 is consistent with communication being a meaningful cost in TP2, but it does not prove that communication caused any M4 crossover. No serving cell was instrumented with a complete causal decomposition into all-reduce, scheduler, kernel and client components. The scientific use of M3 is contextual: it demonstrates an independently measured two-rank communication regime on the tested topology and supplies a concrete, repeatable systems measurement against which mechanistic hypotheses can be discussed cautiously.

Table 6: M3 two-rank NCCL all-reduce fit; intercept and beta are configuration-specific effective terms.

Measure	Estimate
Payload sizes	10
Fitted intercept (microseconds)	112.135079
Intercept 95% CI (microseconds)	93.111210 to 131.158948
Effective beta (GB/s)	4.063916
Beta 95% CI (GB/s)	4.051054 to 4.076860
R squared	0.999984836
RMSE (microseconds)	20.136460
Effective bandwidth at 64 MiB (GB/s)	4.030741

9 M4: repeated multi-model TP crossover

9.1 Compatibility gate and terminal design

Five models entered the frozen compatibility gate. Qwen, Llama 3.2 3B, Phi-4 mini, and Ministral 3 3B reached TP1 and TP2 serving readiness. Gemma 3 4B failed before readiness under the tested FP16/SM75 dtype intersection. It is excluded from principal performance comparison because a throughput value cannot be measured for a configuration that does not reach

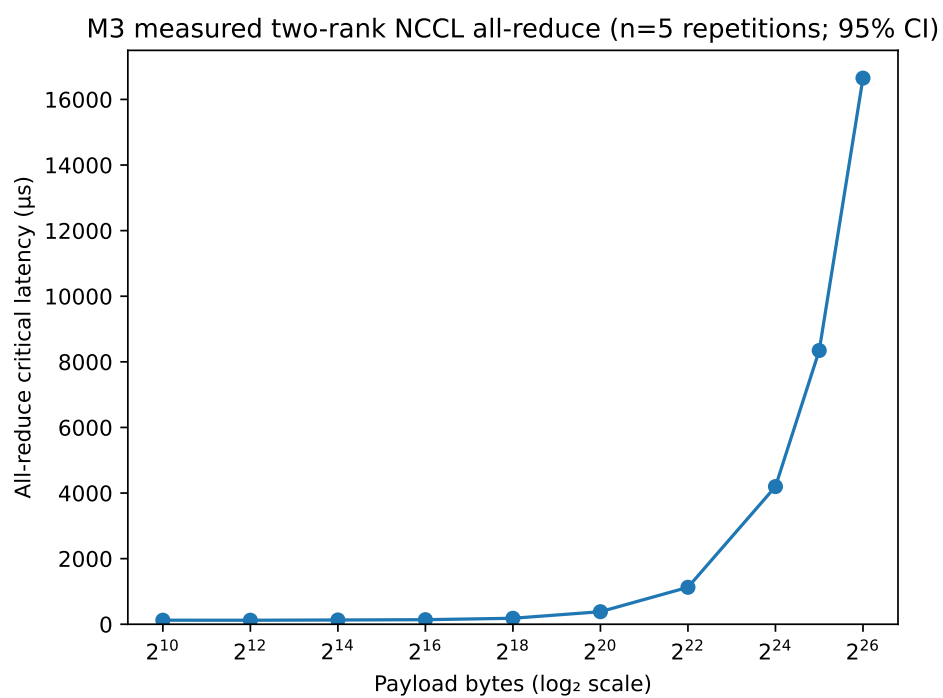


Figure 4: M3 measured all-reduce latency and fitted communication curve; copied canonical figure.

serving. This is a narrow negative compatibility result, not a substitution of another Gemma model and not a claim about every possible precision, kernel or version.

The four eligible models produced 60 terminal logical shards: 55 canonical performance outcomes and five reviewed resource-gated outcomes. The design contained 720 planned fresh-server cells, and the reconciliation records zero unexecuted planned shards. A logical shard is one model–workload–repetition combination containing the TP/concurrency comparisons; a fresh-server cell is a particular TP/concurrency execution within it. Neither the hundreds of client requests nor the 720 cells are the independent N for a model/workload crossover: the inferential comparison uses five matched logical repetitions. This distinction prevents false precision in a dense serving matrix.

9.2 Balanced workload

The balanced shape uses 512 input and 256 output tokens. Llama and Ministral first meet the sustained favorable output-throughput criterion at C=16. Phi first meets it at C=32. Qwen reaches it at C=64. These are first sustained points within the six tested concurrency values, not continuous thresholds between grid points. They also do not imply that every lower point has identical behavior. For example, a positive descriptive TP2/TP1 ratio can be inconclusive when its paired-repetition interval still includes zero. The canonical `m4_crossover_cells.csv` retains means, paired counts, performance classification and latency fields for every balanced cell; Table 7 reports the predeclared final summary.

The model ordering under this single exact-token shape is itself a main result. All four run on the same class of pinned dual-T4 upstream-vLLM stack, yet their favorable points differ. A model-specific computation and memory profile may alter how much useful work TP2 exposes relative to overhead, but this experiment does not isolate any architectural feature as the causal variable. We therefore report the association with model identity rather than a universal explanation for the ordering. Mean output throughput and client latency provide complementary views: the accepted summary’s latency crossover field is recorded separately even where it coincides with the throughput field.

9.3 Prefill-heavy workload

The prefill-heavy shape uses 2048 input and 128 output tokens. Llama, Phi and Ministral all first meet the sustained favorable TP2 output-throughput

rule at $C=4$, earlier than in their balanced and short shapes. Larger prompt work plausibly gives TP2 more parallel computation per request and changes the amortization balance, but that is an interpretation consistent with the pattern, not a measured decomposition of execution time. The result remains conditional on model, exact token count, runtime, and offered concurrency.

Qwen’s prefill-heavy row requires a different statement. The measured cells at $C=8$, 16 and 32 carry favorable throughput classifications. At $C=64$, however, all five TP2 logical repetitions reached the terminal frozen resource boundary. The sustained rule cannot assign a valid throughput crossover that extends through $C=64$, because that endpoint lacks valid TP2 performance. This is not evidence that TP2’s $C=64$ throughput fell below TP1, and it is not a zero-throughput observation. The table therefore leaves the final prefill-heavy Qwen throughput crossover blank and records the $C=64$ resource boundary. Keeping these fields separate is necessary to avoid turning capacity into speed.

9.4 Short/decode-heavy workload

The short shape uses 128 input and 64 output tokens. Llama and Ministral first meet the sustained favorable criterion at $C=32$; Phi reaches it only at $C=64$. Qwen has no sustained favorable output-throughput point within the tested range. This relative ordering is consistent with limited per-request useful work making TP2 overhead harder to amortize, but the data alone cannot apportion the cost among all-reduce, worker scheduling, launch or client effects. The result is particularly relevant to deployments dominated by short exchanges: the second physical GPU can increase usable memory while not improving output throughput at a given low or moderate concurrency.

9.5 Latency behavior and synthesis

The canonical M4 analyzer reports output-throughput and latency crossover fields separately, and its cell table provides TP1/TP2 TTFT, TPOT, ITL and E2E values. The frozen latency crossover field in the 12 model-workload summaries coincides with the sustained throughput field for the measurable rows, but the metric curves and levels differ. TTFT represents prompt and queue effects seen by the client; TPOT and ITL emphasize generated-token pacing; E2E also includes waiting and client overhead. The selected latency figures should therefore be read as metric-specific results rather than as multiple independent demonstrations of one mechanism. In particular, an improvement in one aggregate at high concurrency does not imply that an

isolated request becomes faster on TP2.

Across the final table, the principal pattern is model-workload-concurrency dependence under a resource envelope. Three fully measurable non-Qwen models favor TP2 earliest for prefill-heavy work, later for balanced work, and latest for short work. Qwen shows a late balanced point, no short point, and a prefill-heavy high-concurrency resource boundary. The PHB/NCCL evidence supplies a plausible system context for overhead, but no experiment here varies the interconnect while holding everything else fixed. The main conclusion is thus empirical and scoped: on this tested pinned stack, TP2 benefit is conditional rather than universal.

9.6 Qwen resource boundary as a first-class result

All five Qwen prefill-heavy TP2/C=64 repetitions exceeded the frozen 14,848 MiB per-physical-GPU guard. The canonical resource table records a mean sampled maximum of 14,895 MiB. Those units and the per-GPU comparison matter: the number is not aggregate memory across both devices. The outcome is `RESOURCE_GATED`. It does not by itself prove CUDA OOM, and it yields no valid TP2 output-throughput, request-throughput or latency estimate for that cell. The paper assigns N/A, never zero, to its performance fields.

The resource-gated row limits the performance claim at the very point where extrapolation might otherwise be tempting. We did not relax the guard, add a sixth repetition, shorten the workload or substitute a new run to obtain a favorable measurement. A deployment decision must consider whether the desired configuration executes within its memory budget before applying a throughput comparison. The resource row in Table 8 is therefore part of the central result rather than a discarded failure.

9.7 Reading the cell-level surface

The sustained summary is deliberately compact; it should be read alongside the full cell table and figures. At low offered concurrency, the model/workload rows generally classify TP2 as unfavorable or inconclusive for output throughput. At intermediate points, the classification can change without yet satisfying the “sustained through all higher points” rule. Qwen prefill-heavy work demonstrates why the summary is more than a search for the first favorable cell: several measured middle points are favorable, but the highest planned point is a resource outcome. Qwen balanced work demonstrates a different distinction: a descriptive rate near equality at an intermediate point does not

automatically pass the paired CI criterion. The canonical analyzer retains both means and paired differences so readers can examine these distinctions rather than relying on a single threshold label.

Figures use the repository’s canonical plotting outputs, not redrawn values with a new smoothing rule. The multi-model crossover plot shows rate regimes; the ratio heatmap highlights model and token-shape dependence; selected latency plots show that the client experiences more than aggregate output rate. The numeric authority remains the machine table and analysis JSON. When a visual mark seems ambiguous, its cell row and terminal state take precedence. This is especially important at boundaries, where a plotting library could otherwise interpolate across missing data and visually imply a measurement that was never accepted.

Table 7: M4 first sustained favorable TP2 output-throughput concurrency under matched-repetition 95% CI criterion.

Model	Balanced	Prefill-heavy	Short
Qwen2.5-3B	64	resource-gated at 64	none
Phi-4-mini	32	4	64
Ministral-3-3B	16	4	32
Llama-3.2-3B	16	4	32

Table 8: M4 terminal resource boundary; performance is N/A, never zero.

Model	Workload	C	TP	Gated reps	Mean peak MiB	Ceiling MiB	Performance
Qwen2.5-3B	prefill_heavy	64	2	5	14895	14848	N/A

10 Supplementary ALLaM-7B Arabic/English serving case study

10.1 Model, runtime and evidence hierarchy

The later PR #29 case studies `humain-ai/ALLaM-7B-Instruct-preview` at immutable revision `a28dd1e67420cde72d3629c8633a974cf7d9c366`. It is not a fifth principal M4 model and cannot increase M4’s repetition count. The retained P2, P3 and P4 notebooks and raw evidence use `kaggle-vllm==0.2.0` with the same upstream native-wheel source lineage. The BF16 checkpoint is requested as FP16, eager execution is enforced, custom all-reduce is disabled, worker multiprocessing uses spawn, and `TRITON_ATTN` is selected on

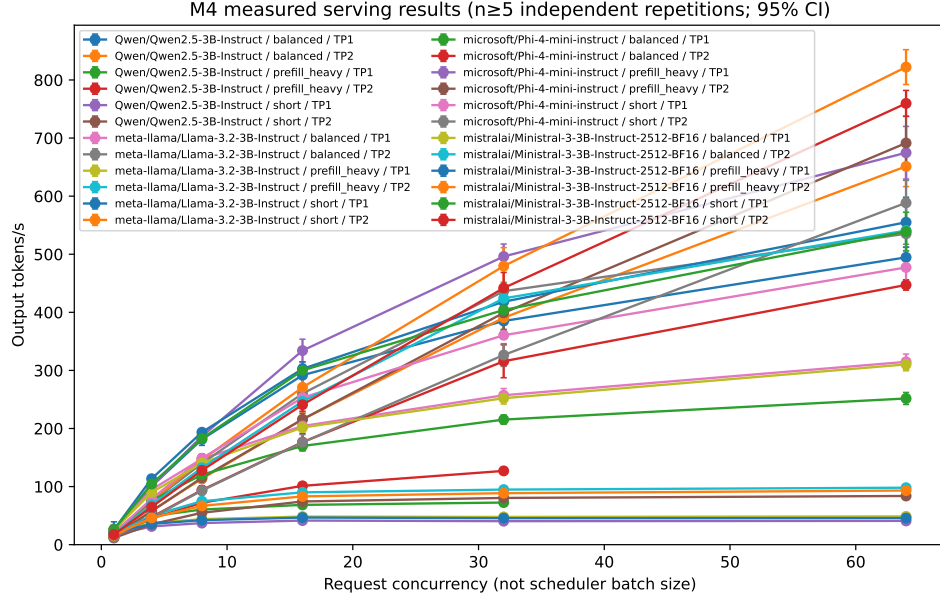


Figure 5: M4 model- and workload-dependent crossover; copied canonical figure.

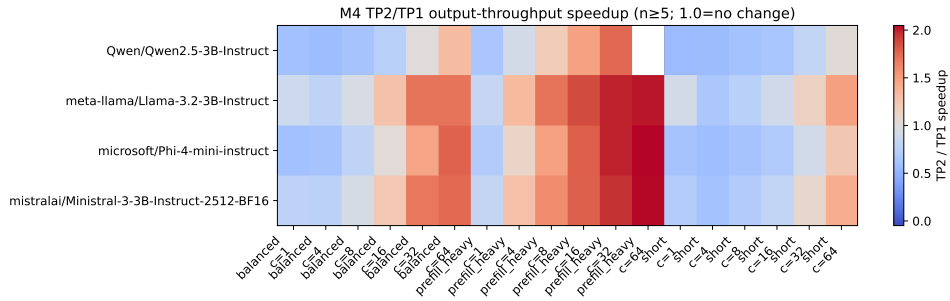


Figure 6: M4 TP2/TP1 output-throughput ratio heatmap; resource cells remain unavailable.

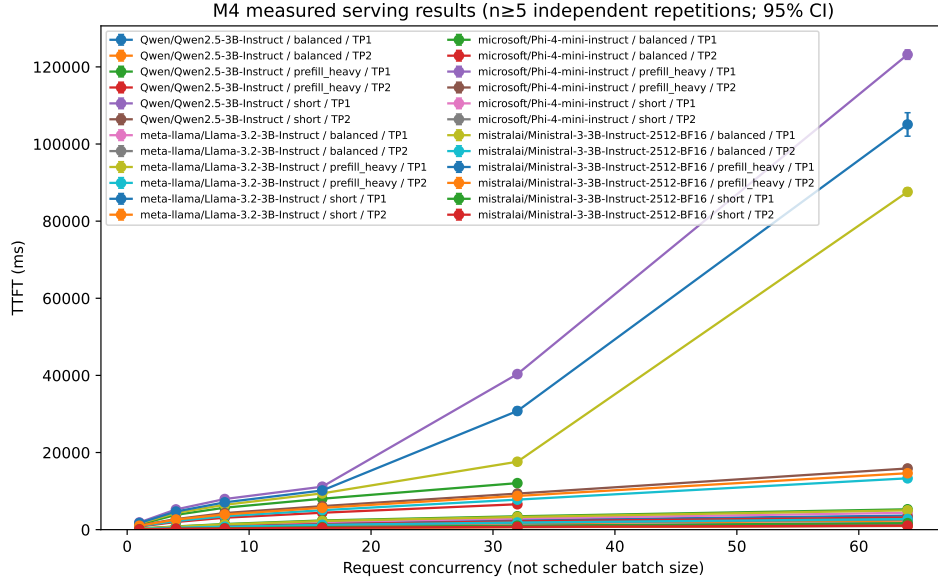


Figure 7: M4 client TTFT comparison; copied canonical figure.

SM75. There is no evidence of successful FlashAttention 2 use on these T4s. The local PR #29 tree and recovered exact merge-commit tarball correspond to the named merge commit.

ALLaM’s original paper addresses Arabic/English model development, tokenization, training and quality evaluation [3]. Its bilingual focus motivates a serving exercise in both languages, but none of its quality scores or A100-scale training claims are results of this dual-T4 inference study. The exact pinned model card is a separate provenance and terms source [4]. The retained P2/P3/P4 observations have different scientific authority: P2 establishes compatibility, P3 is exploratory and composite, and P4 is the strongest online-serving matrix.

10.2 P2 compatibility and P3 exploratory characterization

P2 directly constructed a TP2 engine with maximum model length 2048 and GPU memory utilization 0.85. Its machine result reports PASS, 110.3194 seconds of engine initialization, 7.7651 seconds of bilingual generation, 36 prompt tokens and 17 output tokens in total. Both English and Arabic outputs are nonempty. The worker log records both ranks and NCCL initialization, providing stronger evidence of TP2 execution than mere visibility of two

GPUs. P2 is a smoke and compatibility result; its generation wall time is not an online-serving throughput estimate, and short responses are not a language-quality benchmark.

P3 explored offline `LLM.generate()` behavior under six TP/context configurations. It reused four earlier outcomes and added later executions. Its saved notebook has nonsequential execution counts, and no immutable pre-execution source freeze establishes absence of post-execution editing. At the same time, the retained runner bytes, manifests, raw results, logs and regenerated tables agree. The scientifically conservative classification is an exploratory composite with auditable observations, not a clean newly replicated matrix. In P3, TP1 passed context 960 but resource-gated 2048 and 4096, while TP2 passed all three. P3’s language-mixed, one-wave concurrency sweep is not a controlled English/Arabic crossover test. Its tokenization snippets do not reproduce the original ALLaM paper’s corpus-level tokenizer study.

10.3 P4 capacity matrix

P4 ran a separate loopback OpenAI-compatible server for each of six TP/context configurations. Five reached readiness. TP1 passed context 960 and 2048, resource-gating at 4096; TP2 passed 960, 2048 and 4096. Table 9 reports the six statuses and sampled memory, with N/A performance for the gated configuration. The TP1/4096 server reported 1.0 GiB available KV cache and an estimated maximum model length of 2048, below the requested 4096. Its lower sampled GPU-memory maximum comes from a startup that stopped before serving; that telemetry column is not the capacity-gate criterion and is not directly comparable with a PASS server’s longer lifetime. Across the viable servers, the accepted evidence retains 120 measured waves, 450 successful measured benchmark requests, 10 near-context/capacity probes and 20 chat probes. The CPU-only PR #29 evidence verifier passed against the preserved notebooks, manifests, raw request results and summaries during this audit.

The P3/P4 TP1/2048 disagreement is real and must not be smoothed away. P3’s offline run and P4’s online run differed in batching configuration, available KV-cache memory, and physical hosted GPU allocations/UUIDs. P3’s failed startup records a much smaller available KV-cache allocation than P4’s successful TP1/2048 server. The retained material does not isolate which difference caused the changed boundary. Therefore the result is configuration/execution dependent, not an inherent “2048-token T4 context ceiling.” Near-context probes show that selected long prompts completed

under viable configurations; they do not prove all prompts up to a nominal context limit will succeed.

10.4 Matched-context descriptive online performance

The fair P4 TP1/TP2 performance comparison uses common context 960. Each viable server warmed both languages, then ran English and Arabic separately at client concurrency 1, 2, 4 and 8, with three sequential waves per point. Each wave launched C asynchronous requests with 128 actual prompt and 32 actual output tokens. Wave output throughput is completed output tokens divided by whole-wave wall time. The published rate is the arithmetic mean of three wave rates. TTFT is request start to first nonempty streamed text chunk; E2E includes client/HTTP completion; TPOT is a client-derived average, $(E2E - TTFT) / (\text{completion_tokens} - 1)$. These are not direct CUDA timing measurements.

At C=1 the matched-context mean output rates are close: English TP1/TP2 are 17.2264/16.9932 tokens/s, and Arabic are 16.9513/17.0152. At C=8, English rates are 83.4374/112.2769 and Arabic are 81.2320/111.5477 tokens/s. Arithmetic from the canonical means gives TP2 advantages of 34.56% and 37.32% at C=8 for English and Arabic, respectively. Table 10 provides all four concurrency points for each language. P4 also records lower client p50 E2E and TPOT at higher concurrency under this setup, but the small pooled request counts and fixed execution order constrain tail interpretation.

The three waves at each point share one server lifetime and physical allocation. They are descriptive within-server repeats, not three independent fresh-server sessions. We do not apply M4’s five-repetition 95% inference criterion to ALLaM or claim a universal ALLaM crossover threshold. P4 supports a narrower statement: under its frozen configuration and matched context, TP1 and TP2 output rates are similar at C=1, while TP2’s mean rate is higher at C=8. Capacity is distinct: TP2/4096 passes where TP1/4096 resource-gates in P4. Arabic and English responses show bilingual execution and serving measurements, not comparative language quality. ALLaM remains a bounded case study of performance and capacity regimes, separate from the principal M4 population.

10.5 ALLaM measurement and execution caveats

The online client reports throughput from actual completion-token counts rather than requested maximum tokens, and all measured P4 benchmark

requests in the accepted summary succeeded. The client defines TTFT using the first nonempty streamed text chunk, so an empty transport chunk is not counted as a first token. TPOT is computed from elapsed client times and completion count; it should not be mistaken for per-kernel decode time. Tail percentiles pool the successful requests from the three waves at a point and therefore combine correlated observations from one server lifetime. At C=1, very few requests contribute to a tail estimate, limiting its practical resolution. These choices are suitable for a transparent descriptive case study, but they do not supply a new M4-style inferential population.

P4’s viable servers include warmup and near-context probes alongside the measured online waves. The probes test selected paths and help distinguish capacity from rate, but they are not extra benchmark repetitions. Prefix caching remains enabled in the P4 server; unique early request prefixes reduce deliberate reuse without proving an absence of cache hits. TP1 and TP2 also use different frozen GPU-memory-utilization budgets. Consequently the observed capacity and throughput differences belong to the complete P4 configurations, not to tensor parallel degree in isolation. The server logs retain process-group teardown warnings in some TP1 runs; return codes and successful measured requests support the reported performance, while warning-free shutdown is not claimed.

Table 9: ALLaM P4 capacity. Peaks are sampled telemetry; TP1/4096 failed the KV-cache length startup check.

TP	Context	Status	GPU 0 peak MiB	GPU 1 peak MiB
1	960	PASS	14671	0
1	2048	PASS	14799	0
1	4096	RESOURCE_GATED	13743	0
2	960	PASS	12871	12871
2	2048	PASS	12997	12997
2	4096	PASS	13001	13001

11 Discussion

11.1 Computation, communication, and model dependence

The joint dependence of the crossover on model, token shape and concurrency is compatible with an overhead-amortization account. TP2 partitions work and memory, but a sequence of collectives, worker synchronization, scheduling and launches must be paid even when a request offers little parallel compute. Longer prompt work and larger concurrent batches can expose enough useful

Table 10: ALLaM matched context 960 output throughput: three descriptive within-server wave means per point.

Language	C	TP1 tok/s	TP2 tok/s	TP2/TP1
english	1	17.2264	16.9932	0.986
english	2	25.0009	24.7742	0.991
english	4	51.4014	62.0848	1.208
english	8	83.4374	112.2769	1.346
arabic	1	16.9513	17.0152	1.004
arabic	2	29.0559	31.3422	1.079
arabic	4	50.5218	60.9700	1.207
arabic	8	81.2320	111.5477	1.373

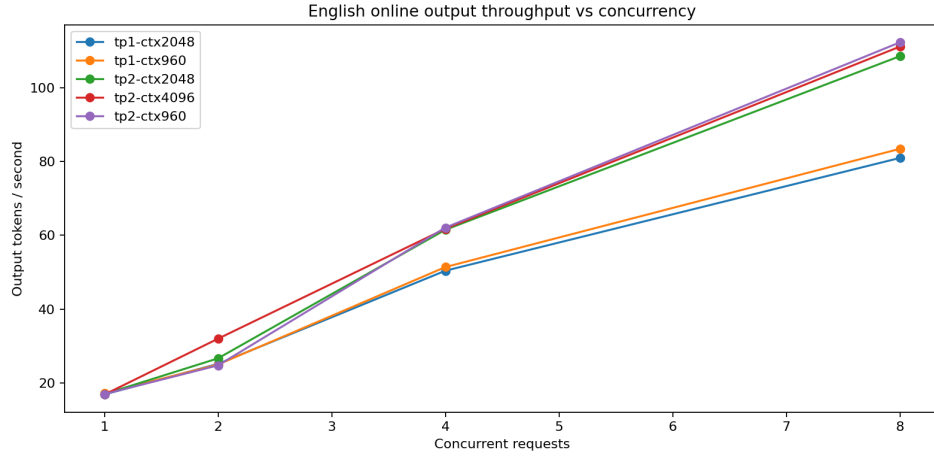


Figure 8: ALLaM P4 descriptive English output throughput at matched context; canonical PR #29 plot.

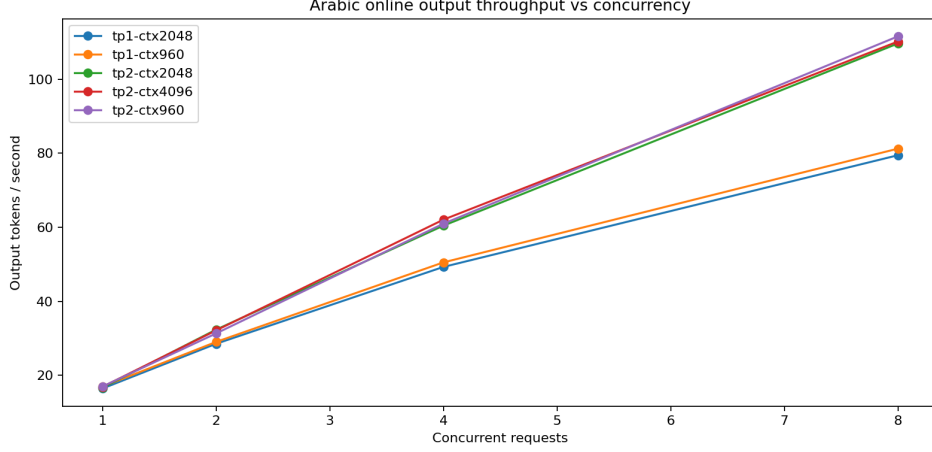


Figure 9: ALLaM P4 descriptive Arabic output throughput at matched context; canonical PR #29 plot.

work to make that cost worthwhile. M1’s small-model low-load disadvantage, M2’s rising Qwen throughput, and M4’s early prefill-heavy crossings point in the same broad direction. They do not establish one unique mechanism. An explanation that attributes the exact favorable point solely to PHB bandwidth, for example, would exceed the retained measurements.

M3 supplies one concrete communication characterization. Its effective large-payload rate and nonzero fitted intercept make it unreasonable to assume that a two-rank collective is free. Yet serving layers call collectives with payloads, ordering, overlap and synchronization patterns different from a standalone microbenchmark. vLLM’s attention, KV-cache and batching behavior also changes with model architecture and request mix [5]. We did not instrument per-layer collective time, vary the interconnect, or run a controlled ablation of scheduler and worker overhead. Mechanistic statements in this paper are therefore hypotheses consistent with end-to-end and independent measurements, not causal estimates.

The model spread is a guard against a single-checkpoint rule. On the same principal grid, the three fully measurable non-Qwen models have similar workload ordering but different balanced and short first favorable points. Qwen’s balanced point is later, its short point absent within the range, and its prefill-heavy high-concurrency endpoint unavailable because of the resource guard. Architecture, implementation, selected kernels, model size, tokenizer/prompt details and memory occupation are entangled. Merely

matching approximate parameter counts does not hold these factors constant. The appropriate deployment takeaway is to measure the actual model and traffic shape under the intended runtime, rather than to import a threshold from a nearby checkpoint.

11.2 Throughput, latency and capacity are different decisions

Output throughput answers how many generated tokens the server completes per wall-clock second for the offered work. Request throughput answers how many requests complete; because output length is fixed in M4, the two are related but not identical in more general settings. TTFT matters when a user waits for the first visible token, whereas TPOT and ITL reflect pacing after generation starts. E2E includes queueing and client overhead. At high concurrency, an aggregate token-rate gain can coexist with a large TTFT or E2E value for individual requests. This paper therefore reports the latency panel and does not equate a favorable output-throughput classification with universal latency improvement for every request or percentile.

Capacity is prior to performance. A TP2 split can enable a context or model configuration that TP1 cannot hold, as P4 ALLaM demonstrates for context 4096 under its particular server settings. Conversely, a TP2 run can hit a memory guard at high concurrency, as Qwen prefill-heavy C=64 demonstrates. These are different statements from “TP2 is faster.” For deployment, one must first establish compatibility with a pinned runtime, then verify the memory envelope, then measure performance on valid cells, and separately assess response quality if quality matters. A resource-gated record is useful operational evidence even though it cannot be plotted as a valid throughput sample.

11.3 Implications for reproducible systems evaluation

A dense benchmark matrix can appear statistically large while having only a few independent experimental units. Here the client request ledgers are detailed but nested within server cells and logical repetitions. The five matched M4 repetitions support the specified paired comparison; the three ALLaM waves on one server do not support the same inference. Reporting this unit explicitly makes the 95% criterion interpretable and prevents request-level pseudoreplication. It also explains why M2’s single matrix cannot determine the final Qwen threshold merely because it contains many successful requests.

The project also illustrates why operational artifacts matter. The pinned wheel and checksum, exact checkpoint revisions, source freezes, terminal

reconciliation, raw ledgers, canonical analysis JSON, and byte-identical figure/table copies establish a path from a claim back to an execution. Recorded irregularities—r03 Ministral recovery, the V19 pre-bootstrap Llama abort, P3’s composite notebook, and the recovered October ZIP’s outer-hash discrepancy—set limits on what can be asserted. A polished narrative cannot replace those records. The aim is a paper that can be challenged and rechecked from evidence, not a claim that all experimental conditions were perfectly controlled.

12 Threats to validity

Platform and external validity. The study uses one hosted dual-T4 class with PHB topology and a pinned CUDA/PyTorch/vLLM stack. It does not sample PCIe generations, NVLink, more than two ranks, multi-node networking, other GPU architectures or independent clouds. Hosted allocations can vary in unmeasured ways across sessions. The M3 fit is tied to its process and NCCL configuration. No universal hardware scaling law follows from this one topology.

Workload and client validity. M4 uses synthetic exact-token prompts and three fixed shapes. Real traffic has variable prompt/output lengths, arrival patterns, prefix reuse, cancellation, and mixed models. The repository client and its timing definitions are part of the measured system; comparison against a separately implemented client or simulator was deferred as optional M5. The ALLaM P4 client uses streaming chunks and fixed language/configuration order, while M4 uses its own serving protocol. Their rates are not directly pooled. Neither study evaluates semantic or linguistic answer quality.

Statistical validity. M4’s N is five matched logical repetitions per model/workload, with possible correlation from batch/session structure and shared hosted environments. The canonical 95% paired-mean rule captures one uncertainty dimension, not every source of variance. The concurrency grid is finite; a first sustained point is only a tested grid point. Multiple model/workload/metric comparisons invite exploratory reading of secondary patterns, so the predeclared output-throughput criterion is emphasized. Individual request counts must not inflate N. M2 is a single matrix. M3’s 5,000 timed collectives are nested within five fresh-process repetitions and ten payloads; a large raw timing count is not 5,000 independent systems. ALLaM’s three waves per point are within-server descriptive repeats, not independent session replications.

Measurement and causal validity. Resource telemetry is sampled and may miss instantaneous peaks. TTFT, TPOT, ITL and E2E are client-derived rather than a complete GPU timeline. M3 all-reduce measurements and M4 serving measurements are separate experiments; their correlation cannot isolate cause. A positive output-throughput ratio without favorable paired CI is not a robust crossover under the declared rule. A resource-gated cell is missing performance by design and cannot be assigned zero. The Gemma result is limited to the frozen FP16/SM75 compatibility intersection.

Provenance validity. The M4 r03 Ministral notebook required a documented post-execution provenance recovery, and the V19 Llama continuation encountered a pre-bootstrap integrity abort; final accepted results rely on reviewed correction and reconciliation rather than erasing those events. ALLaM P3 reused earlier outcomes and lacks a pre-execution source freeze, so absence of notebook edits cannot be proven. P3 and P4 context outcomes differ under distinct configurations and physical allocations. The October evidence bundle, executed collector, runtime JSON and exact PR #29 tarball were recovered and audited after the earlier missing-source finding. The recovered outer ZIP differs from the notebook-reported ZIP by four bytes and has a different SHA256; all 83 internally checksummed files verify. This remaining container-identity warning does not change the historical or ALLaM results.

13 Ethics, licensing and artifact boundaries

The research evaluates compatibility, capacity and performance, not generated-answer quality. Its exact-token prompts and brief bilingual smoke output do not measure truthfulness, safety or linguistic competence. A paper reader should not infer that a faster configuration is a better language model. The ALLaM paper’s Arabic/English training and quality work is cited for model provenance and bilingual motivation only [3]. Its large-scale training system and reported quality evaluations are outside the dual-T4 inference experiment.

Software and model artifacts have separate terms. kaggle-vllm source and upstream vLLM source are distinct distributions; the paper project carries neither a native wheel nor model weights. The principal checkpoints also have distinct pinned model-card terms: the repository metadata classifies Qwen under the Qwen Research License, Llama under the Llama 3.2 Community License, Phi under MIT, and the specified Ministral BF16 checkpoint under Apache-2.0 [10, 6, 7, 8]. The exact ALLaM card declares Apache-2.0 but references a standalone LICENSE not found at its pinned root in the PR

#29 technical review [4]. This is an artifact classification and a disclosure of source state, not legal advice. No redistribution of checkpoint or tokenizer assets is performed here.

The historical Qwen sharded-state archive is intentionally excluded despite a verified checksum. It contains model-state and tokenizer material and is not needed to reproduce the paper’s analysis tables. The paper project includes numeric tables, selected canonical figures, small provenance metadata, scripts and manuscript source. It records hashes and paths for larger raw evidence that remains in the original repository. No credentials or secret tokens are part of the paper files. No official Kaggle or vLLM endorsement, production readiness, venue acceptance or ACM artifact badge is claimed.

14 Reproducibility and artifact availability

The new paper project is a sibling of the original `kaggle-vllm` checkout and leaves that checkout read only. Its `audit/SOURCE_INVENTORY.md` records the checkout HEAD and pre-existing untracked paths; `supplement/provenance/original-git-inspection.txt` retains the exact requested git command outputs. Source paths and SHA256 values for every copied canonical table and figure appear in `audit/TABLE_PROVENANCE.csv` and `audit/FIGURE_PROVENANCE.csv`. Copied files are byte-identical, and `scripts/verify_sources.py` checks both source and destination hashes, source manifests, JSON parseability and required CSV columns. `scripts/verify_claims.py` checks the terminal M4 state, sustained crossover rows, Qwen resource N/A condition, M2 exploratory first advantage and ALLaM P4 counts against accepted machine data. `scripts/generate_tables.py` deterministically produces the LaTeX tables from copied CSV, rather than embedding a second manually maintained set of measurements.

The original repository contains M1–M4 raw evidence and review material under `artifacts/` and `research/`. The local PR #29 worktree and recovered exact merge-commit tarball retain P2/P3/P4 notebooks, manifests, logs, telemetry and raw online request evidence. The October source hashes, archive integrity and executed notebook are recorded in `audit/KAGGLE_EVIDENCE_AUDIT.md` and `audit/OCTOBER_NOTEBOOK_AUDIT.md`. The strict October runtime profile passed; raw two-rank NCCL and TP1/TP2 generation smoke tests passed with return code 0. Both smoke logs contain a nonfatal client ERROR line after generation and engine shutdown. These are post-study runtime checks under driver 580.178.04, not M4 repetitions or independent external replication. The historical benchmark environment

remains the scientific input.

Reproducing the numerical paper tables requires only Python’s standard library and the copied canonical CSV/JSON records. Re-executing the GPU matrix requires the original models, Kaggle environment and frozen protocol and is outside this paper-construction task. This project performs no new GPU experiment and makes no independent-replication claim. The LaTeX source uses a neutral `article` class so a venue template can be selected later without altering the scientific content. A local PDF build requires a LaTeX toolchain; its availability and actual build status are recorded in `STATUS.md`.

15 Conclusion

The accepted M4 evidence does not support a universal TP2 speedup on the tested dual-Tesla-T4 upstream-vLLM stack. The first sustained favorable output-throughput point changes with model, exact-token workload and offered concurrency, and one Qwen endpoint is constrained by a frozen per-GPU resource guard rather than a measured performance reversal. Three fully measurable non-Qwen models cross earliest for prefill-heavy work and later for shorter work. Qwen’s balanced point is late and its short shape has no sustained favorable point within the tested range. The independent NCCL/PHB measurement makes communication overhead concrete but does not identify the cause of the serving pattern. A separate ALLaM-7B case study shows that higher-concurrency descriptive throughput and context capacity can favor TP2 even when low-load output rates are similar. These are conditional, reproducible systems observations, bounded by the pinned runtime, client, topology, prompts and retained experimental units.

A Exact environment, model and workload records

Historical machine-readable environment fields are in `tables/csv/hardware_runtime.csv` and `supplement/provenance/M3_ENVIRONMENT.json`. The latter records driver 580.159.04, CUDA toolkit 12.8.93, two Tesla T4 SM75 devices and the 14,848 MiB M4 per-device guard. The full model revisions are in `tables/csv/m4_principal_model_metadata.csv`; the exact token shapes are in `tables/csv/workload_matrix.csv`. The recovered October collector is separately recorded in `tables/csv/poststudy_runtime_validation.csv` and does not overwrite historical values.

Table 11: Complete immutable M4 checkpoint identities.

Field	Value
Qwen2.5-3B	Qwen/Qwen2.5-3B-Instruct
Revision	aa8e72537993ba99e69dfaafa59ed015b17504d1
Phi-4-mini	microsoft/Phi-4-mini-instruct
Revision	cfbefacb99257ffa30c83adab238a50856ac3083
Ministral-3-3B	mistralai/Ministral-3-3B-Instruct-2512-BF16
Revision	b6d637bef2393152b3da2b2fde72eecdee30557e
Llama-3.2-3B	meta-llama/Llama-3.2-3B-Instruct
Revision	0cb88a4f764b7a12671c53f0838cd831a0843b95

B Full M4 and ALLaM result records

The complete 72-row M4 model–workload–concurrency comparison table is `tables/csv/m4_crossover_cells.csv`; its 12-row sustained summary is `tables/csv/m4_crossover_summary.csv`, and per-TP resource rows are in `tables/csv/m4_resource_summary.csv`. The Qwen prefill-heavy TP2/C=64 comparison has N/A performance and five resource-boundary repetitions. The ALLaM P4 capacity matrix, full performance table, near-context probes and wave summary are copied as `tables/csv/allam-*` files. ALLaM’s matched context 960 rate table is descriptive and should not be pooled with M4.

C Claim and provenance controls

`audit/CLAIM_LEDGER.csv` maps numerical and status claims to machine fields and authority levels. `audit/LIMITATIONS_LEDGER.md` records the principal validity boundaries. `audit/EXPERIMENTAL_AUTHORITY.md` resolves M2/M4 Qwen, ALLaM P3/P4 context, and historical/post-study runtime disagreements without changing any measurement. Figure and table provenance CSVs record original and copied SHA256 digests. The historical Qwen sharded-state archive was verified in place and is referenced only as reload capability evidence; no model-state bytes are included here.

References

- [1] ACM SIGPLAN. Sigplan program chair guidelines: Artifacts and badges. Official ACM SIGPLAN guidance. Accessible official ACM SIGPLAN guidance on artifact evaluation and badges; accessed 2026-10-02.

- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, 2024.
- [3] M Saiful Bari, Yazeed Alnumay, Norah A. Alzahrani, Nouf M. Alotaibi, Hisham A. Alyahya, Sultan AlRashed, Faisal A. Mirza, Shaykhah Z. Alsubaie, Hassan A. Alahmed, Ghadah Alabduljabbar, Raghad Alkhathran, Yousef Almushayqih, Raneem Alnajim, Salman Alsubaihi, Maryam Al Mansour, Majed Alrubaian, Ali Alammari, Zaki Alawami, Abdulmohsen Al-Thubaity, Ahmed Abdelali, Jeril Kuriakose, Abdalghani Abujaabal, Nora Al-Twairish, Areeb Alowisheq, and Haidar Khan. AL-LaM: Large language models for arabic and english. *arXiv preprint arXiv:2407.15390v1*, 2024.
- [4] Humain AI. ALLaM-7B-Instruct-preview pinned model card. Pinned revision a28dd1e67420cde72d3629c8633a974cf7d9c366; accessed 2026-10-02.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pages 611–626. Association for Computing Machinery, 2023.
- [6] Meta. Llama-3.2-3B-Instruct model card and pinned repository. Pinned revision 0cb88a4f764b7a12671c53f0838cd831a0843b95; accessed 2026-10-02.
- [7] Microsoft. Phi-4-mini-instruct model card and pinned repository. Pinned revision cfbefacb99257ffa30c83adab238a50856ac3083; accessed 2026-10-02.
- [8] Mistral AI. Ministral-3-3B-Instruct-2512-BF16 pinned model repository. Pinned revision b6d637bef2393152b3da2b2fde72eecddee30557e; pinned revision and Apache-2.0 card metadata independently checked against the Hugging Face revision API on 2026-10-02.
- [9] NVIDIA. NCCL user guide: Collective operations. Official documentation. Accessed 2026-10-02.

- [10] Qwen Team. Qwen2.5-3B-Instruct model card and pinned repository. Pinned revision aa8e72537993ba99e69dfaafa59ed015b17504d1; accessed 2026-10-02.
- [11] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, et al. MLPerf inference benchmark. *arXiv preprint arXiv:1911.02549*, 2019.
- [12] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [13] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.
- [14] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024.